ĐẶC TẢ THIẾT KẾ gemm_dma_engine

Mục lục

1 Tổng quan **2**
1.1 Tính năng chính . . . 2
1.2 Tham số . . . 2
1.3 Tín hiệu giao tiếp . . . 2
1.4 Tín hiệu AXI . . . 6

2 Đặc tả thanh ghi **7**

3 Mô tả chức năng **7**
3.1 Tổ chức chức năng . . . 7
3.2 `gemm_dma_agen`: địa chỉ và burst . . . 7
3.3 `gemm_dma_rd`: kênh đọc . . . 9
3.4 `gemm_dma_wr`: kênh ghi . . . 9
3.5 Reset, hiện thực, kiểm chứng . . . 10

1. Tổng quan

1.1. Tính năng chính

DMA chép dữ liệu giữa DRAM và scratchpad theo lệnh của Main FSM. **Kênh đọc** chạy FETCH, LOAD_A, LOAD_B, LOAD_BIAS: đọc DRAM bằng burst AXI và ghi vào SRAM (hoặc trả 16 word descriptor cho FSM). **Kênh ghi** chạy STORE_C: đọc buffer C và ghi DRAM. Hai kênh độc lập hoàn toàn, mỗi kênh tối đa một burst outstanding. Mỗi lệnh là một bản sao 2D: `rows` hàng, mỗi hàng `rwords` word, các hàng cách nhau `stride` byte.

Chức năng

- Với lệnh (base, rows, rwords, stride), word thứ n ($0 \leq n < \text{rows} \cdot \text{rwords}$), r = n / rwords, w = n mod rwords, nằm ở địa chỉ DRAM base + r · stride + 4w và ở địa chỉ scratchpad theo các quy tắc địa chỉ và giao tiếp bên dưới. Mỗi word được chép đúng một lần, theo thứ tự n tăng dần
- Mỗi lệnh kết thúc bằng đúng một xung `o_*_done` hoặc `o_*_error` . Lỗi: `rresp` hoặc `bresp` $\neq$ OKAY, hoặc `rlast` sai vị trí
- DMA không đảo byte, không đổi dữ liệu ; không biết M, N, K hay ý nghĩa của buffer
- Kênh đọc và kênh ghi được phép có burst AXI cùng lúc . AXI4 cho phép hai kênh vì chúng không chia sẻ tín hiệu

1.2. Tham số

Bảng 1: Tham số

Tham số	Mặc định	Ý nghĩa
`MAX_BURST_BEATS`	16	Số beat tối đa mỗi burst, 1 … 256. Nối từ tham số cùng tên của top
`WFIFO_DEPTH`	4	localparam: số word của FIFO dữ liệu ghi. 4 đủ cho 1 beat W mỗi chu kỳ vì SRAM trả dữ liệu sau đúng 1 chu kỳ

1.3. Tín hiệu giao tiếp

Bảng 2: Tín hiệu nội bộ của `gemm_dma_engine`

Tên tín hiệu	Độ rộng	Hướng	Mô tả
Clock, reset			
`clk`	1	Input	Clock của khối. Các thanh ghi cập nhật tại cạnh lên.
`rst_n`	1	Input	Reset tích cực mức thấp; được kích hoạt bất đồng bộ. Cạnh nhả reset được đồng bộ với clock tại top trước khi đưa tới các module.
`i_soft_clr`	1	Input	Xung xóa đồng bộ dài một chu kỳ do Main FSM phát. Xóa trạng thái điều khiển tại cạnh lên; phạm vi xóa dữ liệu được quy định trong mục Reset.
Lệnh kênh đọc			
`i_rd_start`	1	Input	Xung một chu kỳ yêu cầu bắt đầu lệnh đọc DRAM. Chỉ phát khi kênh rảnh. Các tham số của lệnh được chốt tại cạnh lên của chu kỳ có i_rd_start = 1.

Bảng 2: Tín hiệu nội bộ của `gemm_dma_engine` (tiếp theo)

Tên tín hiệu	Độ rộng	Hướng	Mô tả
`i_rd_dst`	2	Input	Chọn đích của lệnh đọc. Giá trị 0: buffer A. Giá trị 1: buffer B. Giá trị 2: buffer BIAS. Giá trị 3: descriptor trả về Main FSM. Giá trị được chốt cùng xung i_rd_start.
`i_rd_base`	32	Input	Địa chỉ byte của hàng đầu tiên trong DRAM cho lệnh đọc DRAM. Địa chỉ phải căn 4 byte; được chốt cùng xung i_rd_start.
`i_rd_rows`	6	Input	Số hàng cần truyền trong lệnh đọc DRAM, từ 1 đến 32. Giá trị được chốt cùng xung i_rd_start.
`i_rd_rwords`	6	Input	Số word 32 bit cần truyền trong mỗi hàng của lệnh đọc DRAM, từ 1 đến 32. Giá trị được chốt cùng xung i_rd_start.
`i_rd_stride`	16	Input	Khoảng cách tính bằng byte giữa địa chỉ đầu hai hàng liên tiếp trong DRAM. Phải là bội của 4 và không nhỏ hơn 4 lần số word mỗi hàng. Được chốt cùng xung i_rd_start.
`i_rd_bank`	1	Input	Chọn bank đích của buffer A, B hoặc BIAS khi nạp dữ liệu; không dùng cho FETCH. Giá trị được chốt cùng xung i_rd_start.
`i_rd_abort`	1	Input	Yêu cầu dừng lệnh đọc DRAM; đây là tín hiệu mức. DMA ghi nhớ yêu cầu trong suốt lệnh, không khởi tạo burst mới và hoàn tất các giao dịch đã cam kết. Abort không tự tạo lỗi.
`o_rd_busy`	1	Output	Báo kênh đọc DRAM đang bận. Bằng 1 từ chu kỳ sau khi nhận i_rd_start đến hết chu kỳ báo done hoặc error; không nhận lệnh mới trong khoảng này.
`o_rd_done`	1	Output	Xung một chu kỳ báo lệnh đọc DRAM kết thúc mà không có lỗi AXI. Không xuất hiện đồng thời với error. Lệnh bị abort vẫn báo done nếu không gặp lỗi.
`o_rd_error`	1	Output	Xung một chu kỳ báo lệnh đọc DRAM kết thúc có lỗi. Lỗi gồm phản hồi R khác OKAY hoặc RLAST sai vị trí. Không xuất hiện đồng thời với done; chỉ báo sau khi giao dịch đang dở đã được xử lý.
`o_desc_valid`	1	Output	Báo có một word descriptor hợp lệ trong chu kỳ này. Chỉ dùng với lệnh FETCH; bên nhận lấy dữ liệu khi valid = 1, không có ready.
`o_desc_idx`	4	Output	Chỉ số word trong descriptor, từ 0 đến 15. Hợp lệ cùng chu kỳ với o_desc_valid.
`o_desc_data`	32	Output	Nội dung 32 bit của word descriptor có chỉ số o_desc_idx. Hợp lệ cùng chu kỳ với o_desc_valid; dữ liệu của FETCH bị lỗi không được dùng để thực thi job.

Bảng 2: Tín hiệu nội bộ của `gemm_dma_engine` (tiếp theo)

Tên tín hiệu	Độ rộng	Hướng	Mô tả
Lệnh kênh ghi			
`i_wr_start`	1	Input	Xung một chu kỳ yêu cầu bắt đầu lệnh ghi DRAM. Chỉ phát khi kênh rảnh. Các tham số của lệnh được chốt tại cạnh lên của chu kỳ có i_wr_start = 1.
`i_wr_fmt`	1	Input	Chọn định dạng dữ liệu trong buffer C. Giá trị 0: INT32, mỗi phần tử chiếm một word. Giá trị 1: INT8, bốn phần tử trong một word. Giá trị được chốt cùng xung i_wr_start.
`i_wr_base`	32	Input	Địa chỉ byte của hàng đầu tiên trong DRAM cho lệnh ghi DRAM. Địa chỉ phải căn 4 byte; được chốt cùng xung i_wr_start.
`i_wr_rows`	6	Input	Số hàng cần truyền trong lệnh ghi DRAM, từ 1 đến 32. Giá trị được chốt cùng xung i_wr_start.
`i_wr_rwords`	6	Input	Số word 32 bit cần truyền trong mỗi hàng của lệnh ghi DRAM, từ 1 đến 32. Giá trị được chốt cùng xung i_wr_start.
`i_wr_stride`	16	Input	Khoảng cách tính bằng byte giữa địa chỉ đầu hai hàng liên tiếp trong DRAM. Phải là bội của 4 và không nhỏ hơn 4 lần số word mỗi hàng. Được chốt cùng xung i_wr_start.
`i_wr_bank`	1	Input	Chọn bank nguồn của buffer C khi thực hiện STORE_C. Giá trị được chốt cùng xung i_wr_start.
`i_wr_abort`	1	Input	Yêu cầu dừng lệnh ghi DRAM; đây là tín hiệu mức. DMA ghi nhớ yêu cầu trong suốt lệnh, không khởi tạo burst mới và hoàn tất các giao dịch đã cam kết. Abort không tự tạo lỗi.
`o_wr_busy`	1	Output	Báo kênh ghi DRAM đang bận. Bằng 1 từ chu kỳ sau khi nhận i_wr_start đến hết chu kỳ báo done hoặc error; không nhận lệnh mới trong khoảng này.
`o_wr_done`	1	Output	Xung một chu kỳ báo lệnh ghi DRAM kết thúc mà không có lỗi AXI. Không xuất hiện đồng thời với error. Lệnh bị abort vẫn báo done nếu không gặp lỗi.
`o_wr_error`	1	Output	Xung một chu kỳ báo lệnh ghi DRAM kết thúc có lỗi. Lỗi xảy ra khi phản hồi B khác OKAY. Không xuất hiện đồng thời với done; chỉ báo sau khi giao dịch đang dở đã được xử lý.
Cổng ghi scratchpad (A, B, BIAS)			
`o_spm_wr_valid`	1	Output	Yêu cầu ghi một word 32 bit vào scratchpad. Ghi được nhận khi valid và ready cùng bằng 1 tại cạnh lên.
`o_spm_wr_buf`	2	Output	Chọn buffer đích cho word ghi. Giá trị 0: A. Giá trị 1: B. Giá trị 2: BIAS; giá trị 3 không hợp lệ.

Bảng 2: Tín hiệu nội bộ của `gemm_dma_engine` (tiếp theo)

Tên tín hiệu	Độ rộng	Hướng	Mô tả
`o_spm_wr_bank`	1	Output	Chọn bank đích cho word ghi. Hợp lệ cùng yêu cầu ghi; bỏ qua khi chỉ có một bank.
`o_spm_wr_addr`	10	Output	Địa chỉ word logic trong buffer đích. A/B dùng miền 0 đến 255; BIAS dùng miền 0 đến 31. Hợp lệ cùng yêu cầu ghi.
`o_spm_wr_data`	32	Output	Word dữ liệu 32 bit cần ghi vào scratchpad. Hợp lệ cùng yêu cầu ghi; DMA giữ nguyên thứ tự byte của dữ liệu DRAM.
`i_spm_wr_ready`	1	Input	Báo scratchpad có thể nhận word ghi. Thiết kế SRAM hiện tại luôn giữ bằng 1; giao dịch vẫn được xác định bởi valid và ready.
Cổng đọc scratchpad (C)			
`o_spm_rd_valid`	1	Output	Yêu cầu đọc một word từ buffer C. Yêu cầu được nhận tại cạnh lên khi valid và ready cùng bằng 1.
`o_spm_rd_bank`	1	Output	Chọn bank nguồn của buffer C. Hợp lệ cùng yêu cầu đọc; bỏ qua khi chỉ có một bank.
`o_spm_rd_addr`	10	Output	Địa chỉ word logic cần đọc trong buffer C. Hợp lệ cùng yêu cầu đọc; miền địa chỉ phụ thuộc bố cục INT32 hoặc INT8 của C.
`i_spm_rd_ready`	1	Input	Báo scratchpad có thể nhận yêu cầu đọc C. Thiết kế SRAM hiện tại luôn giữ bằng 1.
`i_spm_rd_rvalid`	1	Input	Báo word đọc trả về hợp lệ. Xuất hiện đúng một chu kỳ sau yêu cầu được nhận; mỗi yêu cầu có đúng một phản hồi, theo thứ tự.
`i_spm_rd_rdata`	32	Input	Word dữ liệu 32 bit đọc từ buffer C. Bên nhận lấy dữ liệu khi read rvalid = 1; không có ready cho kênh trả dữ liệu.

Giao tiếp

- `i_rd_start` chỉ đến khi `o_rd_busy` = 0 . DMA chốt mọi tham số ở chu kỳ đó; FSM được đổi chúng ngay sau
- `i_rd_abort`, `i_wr_abort` được chốt thành cờ `abort_seen` trong suốt lệnh; xóa khi lệnh kết thúc
- `m_axi_arsize` = `m_axi_awsize` = 3'b010; `m_axi_arburst` = `m_axi_awburst` = 2'b01; `m_axi_wstrb` = 4'b1111; `m_axi_bready` = 1
- Mọi port `m_axi_*` ra lấy từ thanh ghi, trừ `m_axi_rready` (= trạng thái $\wedge$ `i_spm_wr_ready`); không có đường tổ hợp từ `m_axi_*` vào tới `m_axi_*` ra
- `o_spm_wr_valid` = `m_axi_rvalid` $\wedge$ `m_axi_rready` $\wedge$ (beat nằm trong burst) $\wedge$ (dst $\neq$ 3); `o_spm_wr_data` = `m_axi_rdata`. Đường tổ hợp này chỉ đi vào SRAM, được phép

1.4. Tín hiệu AXI

Bảng 3: Tín hiệu AXI

Tên tín hiệu	Độ rộng	Hướng	Mô tả
`m_axi_araddr`	32	Output	Địa chỉ byte đầu tiên của burst đọc. Địa chỉ căn 4 byte. Hợp lệ cùng ARVALID; giữ ổn định khi chờ bắt tay.
`m_axi_arlen`	8	Output	Số beat của burst đọc trừ 1. Hợp lệ cùng ARVALID; số beat thực tế từ 1 đến MAX_BURST_BEATS và không vượt biên 4 KiB.
`m_axi_arsize`	3	Output	Kích thước mỗi beat của burst đọc. Luôn bằng 3'b010, tương ứng 4 byte.
`m_axi_arburst`	2	Output	Kiểu burst đọc. Luôn bằng 2'b01 (INCR); địa chỉ tăng 4 byte sau mỗi beat.
`m_axi_arvalid`	1	Output	Báo địa chỉ và thuộc tính yêu cầu đọc hợp lệ. Giữ bằng 1 và giữ nguyên payload cho đến khi ARREADY = 1.
`m_axi_arready`	1	Input	Báo bên nhận sẵn sàng nhận địa chỉ đọc. Bắt tay xảy ra tại cạnh lên khi ARVALID và ARREADY cùng bằng 1.
`m_axi_rdata`	32	Input	Word 32 bit đọc trả về. Hợp lệ khi RVALID = 1; giữ nguyên trong lúc chờ RREADY.
`m_axi_rresp`	2	Input	Mã phản hồi beat đọc. Chỉ OKAY là thành công; mọi giá trị khác làm lệnh kết thúc có lỗi. Hợp lệ cùng RVALID.
`m_axi_rlast`	1	Input	Đánh dấu beat cuối burst đọc khi RVALID = 1. DMA so sánh vị trí với ARLEN + 1; đến sớm hoặc muộn đều là lỗi.
`m_axi_rvalid`	1	Input	Báo dữ liệu đọc hợp lệ. Giữ valid và payload ổn định đến khi RREADY = 1.
`m_axi_rready`	1	Output	Báo bên nhận sẵn sàng nhận dữ liệu đọc. Giao dịch được nhận tại cạnh lên khi RVALID và RREADY cùng bằng 1.
`m_axi_awaddr`	32	Output	Địa chỉ byte đầu tiên của burst ghi. Địa chỉ căn 4 byte. Hợp lệ cùng AWVALID; giữ ổn định khi chờ bắt tay.
`m_axi_awlen`	8	Output	Số beat của burst ghi trừ 1. Hợp lệ cùng AWVALID; số beat thực tế từ 1 đến MAX_BURST_BEATS và không vượt biên 4 KiB.
`m_axi_awsize`	3	Output	Kích thước mỗi beat của burst ghi. Luôn bằng 3'b010, tương ứng 4 byte.
`m_axi_awburst`	2	Output	Kiểu burst ghi. Luôn bằng 2'b01 (INCR); địa chỉ tăng 4 byte sau mỗi beat.
`m_axi_awvalid`	1	Output	Báo địa chỉ và thuộc tính yêu cầu ghi hợp lệ. Giữ bằng 1 và giữ nguyên payload cho đến khi AWREADY = 1.
`m_axi_awready`	1	Input	Báo bên nhận sẵn sàng nhận địa chỉ ghi. Bắt tay xảy ra tại cạnh lên khi AWVALID và AWREADY cùng bằng 1.
`m_axi_wdata`	32	Output	Word 32 bit cần ghi. Hợp lệ khi WVALID = 1; giữ nguyên trong lúc chờ WREADY.

Bảng 3: Tín hiệu AXI (tiếp theo)

Tên tín hiệu	Độ rộng	Hướng	Mô tả
m_axi_wstrb	4	Output	Mặt nạ byte ghi. Luôn bằng 4'b1111 vì DMA ghi nguyên word 32 bit.
m_axi_wlast	1	Output	Đánh dấu beat cuối burst ghi khi WVALID = 1. Bằng 1 ở đúng beat thứ AWLEN + 1 và giữ nguyên trong lúc chờ WREADY.
m_axi_wvalid	1	Output	Báo dữ liệu ghi hợp lệ. Giữ valid và payload ổn định đến khi WREADY = 1.
m_axi_wready	1	Input	Báo bên nhận sẵn sàng nhận dữ liệu ghi. Giao dịch được nhận tại cạnh lên khi WVALID và WREADY cùng bằng 1.
m_axi_bresp	2	Input	Mã phản hồi burst ghi. Chỉ OKAY là thành công; mọi giá trị khác làm lệnh kết thúc có lỗi. Hợp lệ cùng BVALID.
m_axi_bvalid	1	Input	Báo phản hồi ghi hợp lệ. Giữ valid và payload ổn định đến khi BREADY = 1.
m_axi_bready	1	Output	Báo bên nhận sẵn sàng nhận phản hồi ghi. Giao dịch được nhận tại cạnh lên khi BVALID và BREADY cùng bằng 1. DMA luôn giữ BREADY = 1.

2. Đặc tả thanh ghi

Module không có thanh ghi ánh xạ MMIO. Cấu hình được nhận qua các tín hiệu đầu vào; trạng thái nội bộ, điều kiện chốt và reset được mô tả bên dưới.

3. Mô tả chức năng

3.1. Tổ chức chức năng

Bảng 4: Module con

Module	Instance	Vai trò
gemm_dma_agen	u_agen_rd, u_agen_wr	Chia lệnh thành đoạn liên tục, cắt burst, đếm (r, w) và sinh địa chỉ scratchpad
gemm_dma_rd	u_rd	Kênh đọc: AR, R; ghi SRAM hoặc xuất descriptor; lỗi, abort
gemm_dma_wr	u_wr	Kênh ghi: đọc SRAM theo credit, AW, W, B; lỗi, abort
gemm_dma_wfifo	u_wfifo	FIFO đồng bộ WFIFO_DEPTH × 32, đầu ra có thanh ghi (head)

3.2. gemm_dma_agen: địa chỉ và burst

Bảng 5: Tín hiệu giao tiếp của gemm_dma_agen

Tên tín hiệu	Độ rộng	Hướng	Mô tả
i_load	1	Input	Xung chốt các tham số lệnh mới tại cạnh lên. Được phát cùng chu kỳ start của kênh DMA tương ứng.

Bảng 5: Tín hiệu giao tiếp của `gemm_dma_agen` (tiếp theo)

Tên tín hiệu	Độ rộng	Hướng	Mô tả
`i_base`	32	Input	Địa chỉ byte đầu lệnh trong DRAM, căn 4 byte. Chốt khi i_load = 1.
`i_rows`	6	Input	Số hàng cần truyền, từ 1 đến 32. Chốt khi i_load = 1.
`i_rwords`	6	Input	Số word 32 bit của mỗi hàng, từ 1 đến 32. Chốt khi i_load = 1.
`i_stride`	16	Input	Khoảng cách byte giữa đầu hai hàng. Là bội của 4, không nhỏ hơn 4 lần i_rwords; chốt khi i_load = 1.
`i_wide`	1	Input	Chọn bố cục địa chỉ scratchpad. Giá trị 1: ghép r[4:0] với w[4:0]; 0: ghép r[4:0] với w[2:0]. Chốt cùng lệnh.
`o_has_burst`	1	Output	Bằng 1 khi còn burst chưa phát của lệnh hiện tại. Khi bằng 0, kênh DMA không được khởi tạo burst tiếp theo.
`o_burst_addr`	32	Output	Địa chỉ byte đầu burst kế tiếp trong DRAM. Là đầu ra thanh ghi, hợp lệ từ chu kỳ sau i_load hoặc i_burst_next khi còn burst.
`o_burst_beats`	9	Output	Số beat của burst kế tiếp, từ 1 đến 256. Là đầu ra thanh ghi, đi cùng o_burst_addr; đã xét giới hạn độ dài và biên 4 KiB.
`i_burst_next`	1	Input	Xung báo burst hiện tại đã được lấy để phát. Cập nhật mô tả burst tiếp theo ở chu kỳ sau; không tự tăng bộ đếm word.
`i_word_next`	1	Input	Xung báo một word đã được chuyển. Tăng chỉ số word w; khi hết hàng, đưa w về 0 và tăng hàng r.
`o_spm_addr`	10	Output	Địa chỉ word logic của phần tử đang truyền trong scratchpad. Được tạo từ chỉ số hàng r và word w theo i_wide.
`o_word_idx`	4	Output	Bốn bit thấp của số thứ tự word đang truyền. Với FETCH, đây là chỉ số word descriptor từ 0 đến 15.

Bảng 6: Địa chỉ scratchpad theo đích

Lệnh	i_wide	Địa chỉ word n (r = hàng, w = word trong hàng)
LOAD_A, LOAD_B	0	{r[4:0], w[2:0]} = {i, kw} hoặc {k, jw}
LOAD_BIAS	1	{0, w[4:0]} = j
STORE_C, INT32	1	{r[4:0], w[4:0]} = {i, j}
STORE_C, INT8	0	{r[4:0], w[2:0]} = {i, jw}
FETCH	—	Không dùng SPM; `o_word_idx` = w (0 … 15)

Chia đoạn và cắt burst

- Lệnh liên tục khi `stride` = 4 · `rwords`: một đoạn dài `rows` · `rwords` word bắt đầu ở base. Ngược lại: `rows` đoạn, đoạn r dài `rwords` word bắt đầu ở base + r · `stride`
- Trong một đoạn, tại địa chỉ a, burst có min(số word còn lại của đoạn, `MAX_BURST_BEATS`, $(4096 - a \bmod 4096)/4$) beat . Burst không bao giờ vượt biên 4 KiB và không bao giờ trải qua khoảng hở giữa hai đoạn
- `o_burst_*` là thanh ghi, hợp lệ từ chu kỳ sau `i_load` và từ chu kỳ sau `i_burst_next`. Địa chỉ đoạn tính bằng cộng dồn `stride` (không dùng bộ nhân 32 bit); độ dài đoạn liên tục `rows` · `rwords` $\leq$ 1024 cần bộ nhân 6 $\times$ 6 hoặc cộng dồn

- Bộ đếm (r, w) độc lập với burst: `i_word_next` tăng w; khi w = `rwords` − 1 thì w ← 0, r ← r + 1

> **VÍ DỤ CẮT BURST**
> LOAD_A với M = 32, K = 32 liên tục (`lda` = 32): một đoạn 256 word; với `MAX_BURST_BEATS` = 16 và base căn 4 KiB: 16 burst 16 beat. Cùng lệnh với `lda` = 48: 32 đoạn 8 word, mỗi đoạn một burst 8 beat (nếu không vượt 4 KiB), khoảng hở 16 byte giữa hai hàng không bị đọc. Base = 0x0FF8, một đoạn 256 word: burst đầu 2 beat (tới biên 4 KiB), sau đó 15 burst 16 beat và một burst 14 beat.

3.3. `gemm_dma_rd`: kênh đọc

Bảng 7: Trạng thái kênh đọc

Trạng thái	Làm gì	Sang
RD_IDLE	`o_rd_busy` = 0. `i_rd_start`: chốt dst, bank; `agen.i_load`; `err` ← 0, `abort_seen` ← 0	RD_ADDR
RD_ADDR	`arvalid` = 1, `araddr` = `o_burst_addr`, `arlen` = beats − 1, giữ tới khi `arready`; `exp` ← beats, `got` ← 0	RD_DATA sau bắt tay AR (`agen.i_burst_next`)
RD_DATA	`rready` = `i_spm_wr_ready` (dst = 3: 1). Mỗi beat: `got++`; nếu `got` ≤ `exp`: ghi SRAM hoặc xuất descriptor, `agen.i_word_next`; `rresp` ≠ 0 → `err` ← 1. Xử lý `rlast` theo (`rlast` = 1 ở beat có `got` < `exp`: lỗi, burst kết thúc. Beat có `got` = `exp` mà `rlast` = 0: lỗi, tiếp tục nhận (`rready` = 1) và bỏ mọi beat tới khi `rlast` = 1)	Khi burst kết thúc: RD_ADDR nếu còn burst và ¬`err` ∧ ¬`abort_seen`; ngược lại RD_FIN
RD_FIN	Phát `o_rd_error` nếu `err`, ngược lại `o_rd_done`	RD_IDLE

Kênh đọc

- Tối đa 1 burst outstanding: AR kế tiếp chỉ được phát sau beat cuối của burst trước
- `rlast` = 1 ở beat có `got` < `exp`: lỗi, burst kết thúc. Beat có `got` = `exp` mà `rlast` = 0: lỗi, tiếp tục nhận (`rready` = 1) và bỏ mọi beat tới khi `rlast` = 1
- Lỗi hoặc abort: không phát AR mới; burst đã phát AR được nhận hết . Abort không đổi kết quả: lệnh bị abort kết thúc bằng done nếu không có lỗi
- Dữ liệu beat lỗi vẫn được ghi vào SRAM ; FSM không dùng kết quả của lệnh lỗi
- Thông lượng: mỗi chu kỳ một beat khi slave không chèn bọt; giữa hai burst có đúng 1 chu kỳ RD_ADDR cộng độ trễ AR → R của slave

3.4. `gemm_dma_wr`: kênh ghi

Bảng 8: Trạng thái kênh ghi

Trạng thái	Làm gì	Sang
WR_IDLE	`i_wr_start`: chốt fmt, bank; `agen.i_load` với `i_wide` = (fmt = 0); `err`, `abort_seen` ← 0	WR_NEXT
WR_NEXT	1 chu kỳ: nếu `err` ∨ `abort_seen` ∨ ¬`o_has_burst` thì sang WR_FIN. Ngược lại nạp `aw_pend` ← 1, `w_left` ← beats, `rd_left` ← beats; `agen.i_burst_next`	WR_BURST
WR_BURST	Ba việc song song: (1) `awvalid` = `aw_pend` tới khi `awready`; (2) đọc SRAM: `o_spm_rd_valid` = (`rd_left` > 0) ∧ (`fifo_cnt` + `inflight` < 4); mỗi yêu cầu được nhận: `rd_left--`, `agen.i_word_next`; (3) `wvalid` = FIFO khác rỗng ∧ `w_left` > 0, `wdata` = đầu FIFO, `wlast` = (`w_left` = 1); mỗi bắt tay W: lấy khỏi FIFO, `w_left--`	WR_RESP khi AW xong và `w_left` = 0
WR_RESP	Chờ `bvalid` (`bready` = 1); `bresp` ≠ 0 → `err` ← 1	WR_NEXT
WR_FIN	Phát `o_wr_error` nếu `err`, ngược lại `o_wr_done`	WR_IDLE

Kênh ghi

- `wvalid` không chờ `awready` . Một khi `wvalid` = 1 thì giữ tới bắt tay, vì đầu FIFO chỉ đổi khi bắt tay. `wvalid` được phép về 0 giữa hai beat khi FIFO rỗng
- Burst đã vào WR_BURST là đã cam kết: mọi beat W của nó và phản hồi B đều được hoàn tất, kể cả khi `abort_seen` = 1 . Abort chỉ được xét ở WR_NEXT
- Credit: `inflight` = 1 khi có yêu cầu đọc SRAM được nhận ở chu kỳ trước. `i_spm_rd_rvalid` đẩy `i_spm_rd_rdata` vào FIFO. FIFO không bao giờ tràn (assertion)
- Tối đa 1 burst ghi outstanding: AW của burst sau chỉ phát sau khi nhận B của burst trước
- Độ trễ: `i_wr_start` ở chu kỳ t $\rightarrow$ `awvalid` và yêu cầu đọc SRAM đầu tiên ở t + 2; beat W đầu tiên sớm nhất ở t + 4; sau đó một beat mỗi chu kỳ khi `wready` = 1

3.5. Reset, hiện thực, kiểm chứng

Reset

- `rst_n` = 0: hai kênh về IDLE; mọi `valid` AXI ra, `o_*_busy`, xung done/error, FIFO về rỗng
- `i_soft_clr`: như (`rst_n` = 0: hai kênh về IDLE; mọi `valid` AXI ra, `o_*_busy`, xung done/error, FIFO về rỗng). FSM chỉ phát `soft_clr` khi cả hai kênh đã rảnh , nên soft clear không bao giờ cắt ngang một giao dịch AXI

Hiện thực

- SystemVerilog tổng hợp được, không latch; thanh ghi điều khiển có reset, thanh ghi dữ liệu (địa chỉ, FIFO) không bắt buộc reset
- Số beat của burst được tính bằng so sánh ba giới hạn: số word còn lại trong đoạn, MAX_BURST_BEATS và số word tới biên 4 KiB. Giới hạn cuối bằng `1024-a[11:2]`.
- `MAX_BURST_BEATS` ngoài 1 … 256 bị chặn lúc elaboration

Assertion bắt buộc (SVA)

- `arvalid` $\wedge$ `¬arready` $\rightarrow$ ở chu kỳ sau `arvalid` = 1 và `araddr`, `arlen` không đổi; tương tự AW, W (cả `wdata`, `wlast`)
- `arvalid` $\rightarrow$ `arlen` < `MAX_BURST_BEATS` $\wedge$ `araddr[11:2]` + `arlen` $\leq$ 1023; tương tự AW
- Không có AR mới khi burst đọc trước chưa có `rlast`; không có AW mới khi chưa nhận B của burst trước
- Số beat W giữa hai lần `wlast` = `awlen` + 1 của burst tương ứng
- `i_rd_start` $\rightarrow$ `¬o_rd_busy`; mỗi `i_rd_start` có đúng một `o_rd_done` hoặc `o_rd_error`; tương tự kênh ghi
- Sau khi `abort_seen` = 1, không có `arvalid` hay `awvalid` nào bật lên mới
- FIFO không tràn, không lấy khi rỗng
- `i_soft_clr` $\rightarrow$ cả hai kênh ở IDLE

Bảng 9: Tài nguyên ước tính

Hạng mục	Số lượng
Bộ cộng 32 bit (địa chỉ burst, địa chỉ đoạn)	4 (2 mỗi kênh)
Bộ nhân 6 $\times$ 6 (độ dài đoạn liên tục)	2 (hoặc cộng dồn)
FIFO ghi	4 $\times$ 32 bit
Thanh ghi trạng thái và tham số	khoảng 2 $\times$ 140 bit

> **KIỂM CHỨNG BẰNG MÔ HÌNH**
> `gemm_dma_model.py`: (1) 15 000 lệnh ngẫu nhiên (5 loại, stride liên tục hoặc có hở, `MAX_BURST_BEATS` 1 … 256): chuỗi

> địa chỉ DRAM và scratchpad khớp công thức base + hàng $\times$ stride + 4 $\times$ word, không burst nào vượt 4 KiB hay trải qua khoảng hở; (2) 1 500 lệnh kênh đọc và 1 500 lệnh kênh ghi với slave ngẫu nhiên, gồm lỗi RESP, `rlast` sớm hoặc trễ, abort: dữ liệu đúng khi không lỗi, không AR sau abort, không burst nào bị bỏ dở.